

POSSESSION OF MOBILES IN EXAM IS UFM PRACTICE.

Name Riya Verma

Enrollment No.

23803022

Jaypee Institute of Information Technology, Noida

TI Examination, Odd 2024

B. Tech CSE, Int CSE, III Semester

Course Title: Data Structures

Maximum Time: 01 Hr

Course Code: 15B11CI311

Maximum Marks: 20

COURSE OUTCOMES		COGNITIVE LEVELS
C210.1	Explain abstract data types, memory allocation schemes, and need of linear and non-linear data structures	Understand Level (Level 2)
C210.2	Apply and implement various linear data structures, like array, linked list, stack, and queue in different problems and applications	Apply Level (Level 3)
C210.3	Analyse the performance of various sorting and searching techniques	Analyse Level (Level 4)
C210.4	Demonstrate and implement various operations like search, traverse, insertion, deletion, etc. on different non-linear data structures	Apply Level (Level 4)
C210.5	Apply appropriate data structure to design an efficient solution for given and identified problem	Create Level (Level 6)

Note: Attempt all the questions.

Q.1 Add code in line no. 4, 9 and 12 i.e. dotted lines such that the function reverses the linked list.

[CO 1 (Understanding), 3 Marks]

```

Node * reverseList (LinkedList * l)
1. {
2. Node * a=l->head;
3. Node * b= l->head->next;
4. while(-----)
5. { Node * c=b->next;
6. b->next=a;
7. a=b;
8. b=c;
9. .... }
10. b->next=a;
11. l->head->next=NULL;
12. -----;
13. return l->head;
14. }
    
```

Q.2 Apply insertion sort to sort the content of a singly linked list with the starting node address stored in **head** pointer using a function **insertionSort(LLNode * head)**.

[CO 2(Applying), 5 Marks]

Q.3 A queue data structure when implemented using an array post many shortcomings like false overflow (queue is full even though space is available) or overheads of shifting elements if element is deleted to avoid false overflow. Propose a modified queue data structure in which the front and rear indexes of the array be manipulated for enqueueing (insertion) and dequeuing (deletion) of elements eliminating the overhead of elements shifting altogether.

Write the following enqueue and dequeue functions for the modified queue with overflow and underflow conditions.

- enQ(int * Q, int size, int info) that add integer info at the rear index of the array Q implemented as a Queue.
- int deQ(int * Q, int size) which delete the element from the front index and return it.

[CO 2(Applying), 4 Marks]

Q.4 In context of sorting the array of integer elements {1,2,3,4,5,6,7,8} using quick sort; arrange these elements for the following cases individually:

[CO 3(Analysing), 2 Marks]

- If pivot is always taken as the first element, and quick sort algorithm needs maximum swaps.

- b) If pivot is always taken as the last element, and quick sort algorithm needs minimum swaps.
- c) If pivot is always taken as the mid element, and quick sort algorithm needs minimum swaps.
- d) If pivot is always taken as the first element, and quick sort algorithm needs minimum swaps.

Q. 5 Comprehend the following recursive codes and write the output showing all intermediate steps.

[CO 1(Understanding), 3 Marks]

```
a)
#include<stdio.h>
void abc1(int);
void abc2(int);
void abc1(int num)
{
    if(num==0)
        return;
    printf("%d", num);
    abc2(num-3);
    printf("%d",num);
}
```

```
void abc2(int num){
    if(num==0) return;
    printf("%d",num);
    abc1(-num);
    printf("%d",num);
}
int main()
{
    abc1(7);
    return 0;
}
```

```
b)
#include<stdio.h>
int Xf(int num)
{
    printf("%d\n",
num);
    if(num > 8)
    {
        Xf(Xf(Xf(--num)));
    }
    return num;
}
```

```
int main()
{
    Xf(10);
    return 0;
}
```

Q. 6 Analyse the correctness of the following statements.

[CO3(Analysing), 3 Marks]

- a) Interpolation Search is always better than binary search for sorted array of elements.
- b) Binary search is always better than linear search irrespective of the data on which the search is performed.
- c) For finding 5 largest elements from an array of 1000 elements, selection/bubble sort will require lesser comparisons than merge sort.